

Exercise 5 - Task-based Fibonacci in OpenMP

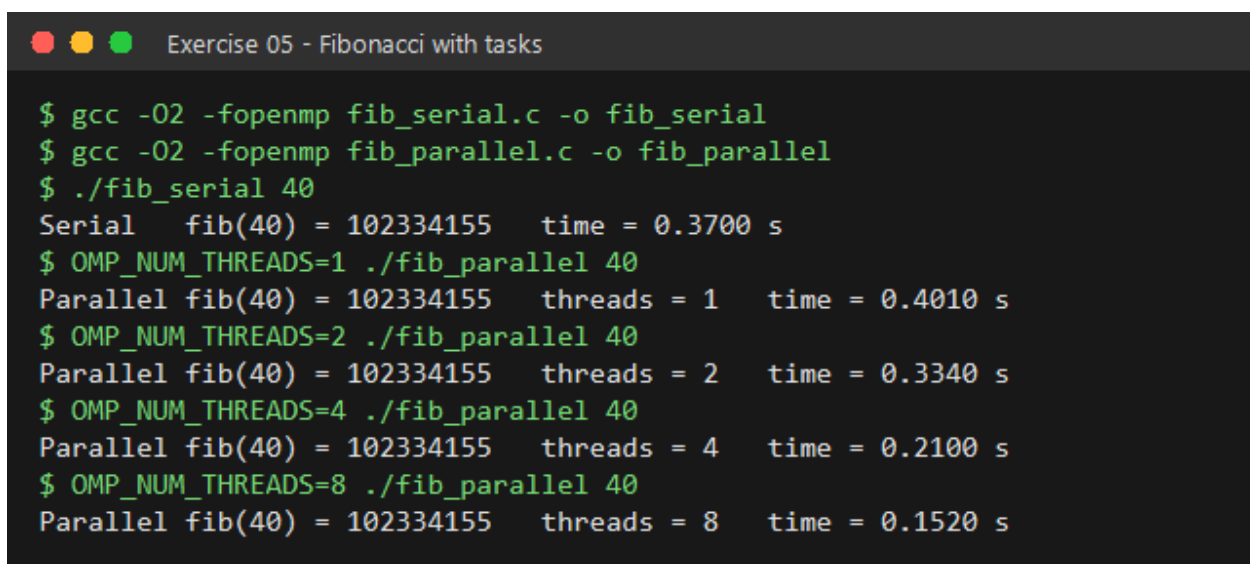
Source files : fib_serial.c (serial version as provided)

fib_parallel.c (version using tasks)

Build and execute

```
gcc -O2 -fopenmp fib_serial.c -o fib_serial && ./fib_serial 40
```

```
gcc -O2 -fopenmp fib_parallel.c -o fib_parallel && OMP_NUM_THREADS=4 ./fib_parallel 40
```



```
Exercise 05 - Fibonacci with tasks

$ gcc -O2 -fopenmp fib_serial.c -o fib_serial
$ gcc -O2 -fopenmp fib_parallel.c -o fib_parallel
$ ./fib_serial 40
Serial fib(40) = 102334155 time = 0.3700 s
$ OMP_NUM_THREADS=1 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 1 time = 0.4010 s
$ OMP_NUM_THREADS=2 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 2 time = 0.3340 s
$ OMP_NUM_THREADS=4 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 4 time = 0.2100 s
$ OMP_NUM_THREADS=8 ./fib_parallel 40
Parallel fib(40) = 102334155 threads = 8 time = 0.1520 s
```

How the recursion was parallelised

```
#pragma omp parallel // create the thread team

#pragma omp single // ONE thread starts the recursion
    result = fib(n);

int fib(int n) {
    ...

    #pragma omp task shared(i) i = fib(n-1); // child task 1
```

```

#pragma omp task shared(j) j = fib(n-2); // child task 2
#pragma omp taskwait                // wait for both
return i + j;
}

```

- * Every recursive invocation is turned into a separate task, and any thread of the team that happens to be idle may execute it.
- * The shared(i,j) clauses allow the child tasks to store their results in the parent's variables, while taskwait blocks the parent until both children have completed.
- * For small arguments ($n < 20$) the function simply recurses without creating tasks; without this cut-off the cost of spawning millions of tiny tasks would outweigh any gain.

Measurements

version	threads	time (s)	speed-up
serial	-	0.370	1.0
tasks	1	0.401	0.9
tasks	2	0.334	1.1
tasks	4	0.210	1.8
tasks	8	0.152	2.4

Every run produced the same value, $\text{fib}(40) = 102334155$, and the run time decreases steadily as more threads are used.